Projeto de Grafos em C

Gerado por Doxygen 1.9.8

1 Índice das estruturas de dados **1**

1.1 Estruturas de dados . . . 1

2 Índice dos ficheiros **3**

2.1 Lista de ficheiros . . . 3

3 Documentação da estruturas de dados **5**

3.1 Referência à estrutura Adjacencia . . . 5

3.1.1 Descrição detalhada . . . 5

3.1.2 Documentação dos campos e atributos . . . 5

3.1.2.1 coluna . . . 5

3.1.2.2 frequencia . . . 5

3.1.2.3 linha . . . 6

3.1.2.4 prox . . . 6

3.2 Referência à estrutura Interferencia . . . 6

3.2.1 Descrição detalhada . . . 6

3.2.2 Documentação dos campos e atributos . . . 6

3.2.2.1 coluna . . . 6

3.2.2.2 linha . . . 6

3.2.2.3 prox . . . 7

3.3 Referência à estrutura Vertice . . . 7

3.3.1 Descrição detalhada . . . 7

3.3.2 Documentação dos campos e atributos . . . 7

3.3.2.1 adjacentes . . . 7

3.3.2.2 coluna . . . 7

3.3.2.3 frequencia . . . 7

3.3.2.4 linha . . . 7

3.3.2.5 prox . . . 7

4 Documentação do ficheiro **9**

4.1 Referência ao ficheiro ficheiros.c . . . 9

4.2 Referência ao ficheiro ficheiros.h . . . 9

4.2.1 Descrição detalhada . . . 9

4.2.2 Documentação das funções . . . 10

4.2.2.1 carregarMatriz() . . . 10

4.2.2.2 gravarGrafoBincomInterferencias() . . . 10

4.2.2.3 lerGrafoBinario() . . . 11

4.3 ficheiros.h . . . 11

4.4 Referência ao ficheiro grafo_funcoes.c . . . 11

4.4.1 Documentação das funções . . . 12

4.4.1.1 calcularInterferencias() . . . 12

4.4.1.2 contarVertices() . . . 13

4.4.1.3 criarAdjacente() . . . 13

4.4.1.4 criarVertice() . . . 13
4.4.1.5 detetarIntersecoes() . . . 14
4.4.1.6 encontrarVertice() . . . 14
4.4.1.7 imprimirInterferencias() . . . 15
4.4.1.8 inserirAdjacente() . . . 15
4.4.1.9 inserirVertice() . . . 16
4.4.1.10 intersecoesParametricas() . . . 16
4.4.1.11 libertarGrafo() . . . 17
4.4.1.12 libertarInterferencias() . . . 17
4.4.1.13 ligarAntenasMesmaFrequencia() . . . 17
4.4.1.14 listarAntenas() . . . 19
4.5 Referência ao ficheiro grafo_funcoes.h . . . 19
4.5.1 Descrição detalhada . . . 20
4.5.2 Documentação das funções . . . 20
4.5.2.1 calcularInterferencias() . . . 20
4.5.2.2 contarVertices() . . . 21
4.5.2.3 criarAdjacente() . . . 21
4.5.2.4 criarVertice() . . . 21
4.5.2.5 detetarIntersecoes() . . . 22
4.5.2.6 encontrarVertice() . . . 22
4.5.2.7 imprimirInterferencias() . . . 23
4.5.2.8 inserirAdjacente() . . . 23
4.5.2.9 inserirVertice() . . . 24
4.5.2.10 intersecoesParametricas() . . . 24
4.5.2.11 libertarGrafo() . . . 25
4.5.2.12 libertarInterferencias() . . . 25
4.5.2.13 ligarAntenasMesmaFrequencia() . . . 25
4.5.2.14 listarAntenas() . . . 27
4.6 grafo_funcoes.h . . . 27
4.7 Referência ao ficheiro grafo_structs.h . . . 28
4.7.1 Descrição detalhada . . . 28
4.7.2 Documentação dos tipos . . . 29
4.7.2.1 Adjacencia . . . 29
4.7.2.2 Interferencia . . . 29
4.7.2.3 Vertice . . . 29
4.8 grafo_structs.h . . . 29
4.9 Referência ao ficheiro main.c . . . 29
4.9.1 Descrição detalhada . . . 30
4.9.2 Documentação das funções . . . 30
4.9.2.1 main() . . . 30
4.10 Referência ao ficheiro procura.c . . . 30
4.10.1 Descrição detalhada . . . 31

4.10.2 Documentação das funções 31
4.10.2.1 bfs() 31
4.10.2.2 dfs() 31
4.11 Referência ao ficheiro procura.h 32
4.11.1 Documentação das funções 32
4.11.1.1 bfs() 32
4.11.1.2 dfs() 33
4.12 procura.h 33

Índice **35**

Capítulo 1

Índice das estruturas de dados

1.1 Estruturas de dados

Lista das estruturas de dados com uma breve descrição:

Adjacencia
: Representa uma ligação entre vértices (antenas) 5

Interferencia
: Representa uma posição de interferência detetada 6

Vertice
: Representa um vértice (antena) da lista principal 7

Capítulo 2

Índice dos ficheiros

2.1 Lista de ficheiros

Lista de todos os ficheiros com uma breve descrição:

ficheiros.c
: Implementação da leitura do ficheiro matriz.txt para construir a lista ligada de vértices 9

ficheiros.h
: Declaração das funções de leitura e escrita do grafo em ficheiros 9

grafo_funcoes.c 11

grafo_funcoes.h
: Declaração das funções de manipulação da lista ligada de vértices 19

grafo_structs.h
: Definição das estruturas de dados usadas para representar grafos 28

main.c
: Ficheiro principal de execução do programa de análise de grafos de antenas 29

procura.c
: Implementação de DFS e BFS para listas ligadas 30

procura.h 32

Capítulo 3

Documentação da estruturas de dados

3.1 Referência à estrutura Adjacencia

Representa uma ligação entre vértices (antenas).

`#include <grafo_structs.h>`

Diagrama de colaboração para Adjacencia:

Campos de Dados

- int linha
- int coluna
- char frequencia
- struct Adjacencia * prox

3.1.1 Descrição detalhada

Representa uma ligação entre vértices (antenas).

-

3.1.2 Documentação dos campos e atributos

3.1.2.1 coluna

`int Adjacencia::coluna`

3.1.2.2 frequencia

`char Adjacencia::frequencia`

3.1.2.3 linha

```
int Adjacencia::linha
```

3.1.2.4 prox

```
struct Adjacencia* Adjacencia::prox
```

A documentação para esta estrutura foi gerada a partir do seguinte ficheiro:

- grafo_structs.h

3.2 Referência à estrutura Interferencia

Representa uma posição de interferência detetada.

```
#include <grafo_structs.h>
```

Diagrama de colaboração para Interferencia:

Campos de Dados

- int linha
- int coluna
- struct Interferencia * prox

3.2.1 Descrição detalhada

Representa uma posição de interferência detetada.

3.2.2 Documentação dos campos e atributos

3.2.2.1 coluna

```
int Interferencia::coluna
```

3.2.2.2 linha

```
int Interferencia::linha
```

3.2.2.3 prox

```
struct Interferencia* Interferencia::prox
```

A documentação para esta estrutura foi gerada a partir do seguinte ficheiro:

- grafo_structs.h

3.3 Referência à estrutura Vertice

Representa um vértice (antena) da lista principal.

```
#include <grafo_structs.h>
```

Diagrama de colaboração para Vertice:

Campos de Dados

- int linha
- int coluna
- char frequencia
- Adjacencia * adjacentes
- struct Vertice * prox

3.3.1 Descrição detalhada

Representa um vértice (antena) da lista principal.

3.3.2 Documentação dos campos e atributos

3.3.2.1 adjacentes

```
Adjacencia* Vertice::adjacentes
```

3.3.2.2 coluna

```
int Vertice::coluna
```

3.3.2.3 frequencia

```
char Vertice::frequencia
```

3.3.2.4 linha

```
int Vertice::linha
```

3.3.2.5 prox

```
struct Vertice* Vertice::prox
```

A documentação para esta estrutura foi gerada a partir do seguinte ficheiro:

- grafo_structs.h

Capítulo 4

Documentação do ficheiro

4.1 Referência ao ficheiro ficheiros.c

Implementação da leitura do ficheiro matriz.txt para construir a lista ligada de vértices.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "grafo_funcoes.h"
```

Diagrama de dependências de inclusão para ficheiros.c:

4.2 Referência ao ficheiro ficheiros.h

Declaração das funções de leitura e escrita do grafo em ficheiros.

```
#include <stdbool.h>
#include "grafo_structs.h"
```

Diagrama de dependências de inclusão para ficheiros.h: Este grafo mostra quais são os ficheiros que incluem directamente ou indirectamente este ficheiro:

Funções

- Vertice * carregarMatriz (const char *nomeFicheiro, int *numLinhas, int *numColunas)

  Lê uma matriz de um ficheiro de texto e constrói uma lista ligada de vértices.
- bool gravarGrafoBincomInterferencias (const char *nomeFicheiro, Vertice *lista, Interferencia *interferencias)

  Grava num ficheiro binário a lista ligada de vértices (antenas) e a lista ligada de interferências associadas.
- bool lerGrafoBinario (const char *nomeFicheiro, Vertice **lista)

  Lê de um ficheiro binário e reconstrói a lista ligada de vértices.

4.2.1 Descrição detalhada

Declaração das funções de leitura e escrita do grafo em ficheiros.

4.2.2 Documentação das funções

4.2.2.1 carregarMatriz()

```
Vertice * carregarMatriz (
            const char * nomeFicheiro,
            int * numLinhas,
            int * numColunas )
```

Lê uma matriz de um ficheiro de texto e constrói uma lista ligada de vértices.

Cada célula diferente de '.' é transformada num vértice com coordenadas e frequência.

Parâmetros

nomeFicheiro	Caminho para o ficheiro .txt com a matriz.
numLinhas	Ponteiro para guardar o número de linhas da matriz.
numColunas	Ponteiro para guardar o número máximo de colunas.

Retorna

Ponteiro para o início da lista de vértices, ou NULL em caso de erro.

Grafo de chamadas desta função: Este é o diagrama das funções que utilizam esta função:

4.2.2.2 gravarGrafoBincomInterferencias()

```
bool gravarGrafoBincomInterferencias (
            const char * nomeFicheiro,
            Vertice * lista,
            Interferencia * interferencias )
```

Grava num ficheiro binário a lista ligada de vértices (antenas) e a lista ligada de interferências associadas.

O ficheiro binário terá o seguinte formato:

- Número total de vértices (int)
- Para cada vértice:
  - Linha (int)
  - Coluna (int)
  - Frequência (char)
  - Número de adjacências (int)
  - Para cada adjacência:
    - Linha (int)
    - Coluna (int)
    - Frequência (char)
- Número total de interferências (int)
- Para cada interferência:
  - Linha (int)
  - Coluna (int)

Parâmetros

nomeFicheiro	Caminho do ficheiro binário onde os dados serão gravados.
lista	Ponteiro para o início da lista ligada de vértices (antenas).
interferencias	Ponteiro para o início da lista ligada de interferências.

Retorna

true se o grafo e as interferências forem gravados com sucesso, false caso contrário.

4.2.2.3 lerGrafoBinario()

```
bool lerGrafoBinario (
            const char * nomeFicheiro,
            Vertice ** lista )
```

Lê de um ficheiro binário e reconstrói a lista ligada de vértices.

Parâmetros

nomeFicheiro	Caminho do ficheiro binário.
lista	Ponteiro para a lista ligada de vértices (por referência).

Retorna

true se a leitura for bem-sucedida, false caso contrário.

Grafo de chamadas desta função:

4.3 ficheiros.h

Ir para a documentação deste ficheiro.

```
00001
00006 #ifndef FICHEIROS_H
00007 #define FICHEIROS_H
00008
00009 #include <stdbool.h>
00010 #include "grafo_structs.h"
00011
00022 Vertice* carregarMatriz(const char* nomeFicheiro, int* numLinhas, int* numColunas);
00023
00049 bool gravarGrafoBincomInterferencias(const char* nomeFicheiro, Vertice* lista, Interferencia*
      interferencias);
00050
00058 bool lerGrafoBinario(const char* nomeFicheiro, Vertice** lista);
00059
00060 #endif // FICHEIROS_H
```

4.4 Referência ao ficheiro grafo_funcoes.c

```
#include <stdio.h>
#include <stdlib.h>
#include "grafo_funcoes.h"
```

Diagrama de dependências de inclusão para grafo_funcoes.c:

Funções

- Vertice * criarVertice (int linha, int coluna, char frequencia)

  Cria dinamicamente uma nova antena (vértice) com os dados fornecidos.
- Vertice * inserirVertice (Vertice *lista, Vertice *novo)

  Insere um vértice no início da lista ligada de vértices.
- Vertice * encontrarVertice (Vertice *lista, int linha, int coluna)

  Procura um vértice(antena) na lista ligada com base nas suas coordenadas.
- int contarVertices (Vertice *lista)

  Conta o número total de vértices numa lista ligada.
- Adjacencia * criarAdjacente (int linha, int coluna, char frequencia)

  Cria dinamicamente uma nova adjacência para um vértice.
- Adjacencia * inserirAdjacente (Adjacencia *listaAdj, Adjacencia *nova)

  Insere uma adjacência no início da lista de adjacências do vértice.
- bool listarAntenas (Vertice *lista)

  Lista todas as antenas presentes na lista ligada de vértices.
- Interferencia * calcularInterferencias (Vertice *lista, int numLinhas, int numColunas)

  Calcula interferências entre pares de antenas com a mesma frequência.
- bool imprimirInterferencias (Interferencia *lista)

  Imprime todas as posições de interferência presentes na lista ligada.
- bool libertarInterferencias (Interferencia *lista)

  Liberta toda a memória alocada para a lista ligada de interferências.
- bool libertarGrafo (Vertice *lista)

  Liberta toda a memória associada à lista ligada de vértices e suas adjacências.
- bool intersecoesParametricas (int x1, int y1, int x2, int y2, int x3, int y3, int x4, int y4, double *ix, double *iy)

  Verifica se dois segmentos se intersectam e, em caso afirmativo, calcula o ponto de interseção usando equações paramétricas.
- bool detetarIntersecoes (Vertice *lista)

  Verifica e imprime todas as interseções entre arestas do grafo.
- bool ligarAntenasMesmaFrequencia (Vertice *lista)

  Liga vértices que têm a mesma frequência, criando adjacências entre eles.

4.4.1 Documentação das funções

4.4.1.1 calcularInterferencias()

```
Interferencia * calcularInterferencias (
            Vertice * lista,
            int numLinhas,
            int numColunas )
```

Calcula interferências entre pares de antenas com a mesma frequência.

Esta função percorre todos os pares de vértices (antenas) na lista e, quando encontra duas antenas com a mesma frequência, calcula os pontos simétricos entre elas, que representam possíveis zonas de interferência. Apenas pontos dentro dos limites da matriz (definidos por numLinhas e numColunas) são considerados válidos.

As interferências detetadas são armazenadas numa lista ligada.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.
numLinhas	Número total de linhas da matriz original.
numColunas	Número total de colunas da matriz original.

Retorna

Ponteiro para o início da lista ligada de interferências, ou NULL se nenhuma interferência for encontrada.

Este é o diagrama das funções que utilizam esta função:

4.4.1.2 contarVertices()

```
int contarVertices (
            Vertice * lista )
```

Conta o número total de vértices numa lista ligada.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.

Retorna

Número de vértices encontrados na lista.

4.4.1.3 criarAdjacente()

```
Adjacencia * criarAdjacente (
            int linha,
            int coluna,
            char frequencia )
```

Cria dinamicamente uma nova adjacência para um vértice.

Esta função representa a ligação entre duas antenas, armazenando as coordenadas e a frequência da antena de destino. O campo `prox` é inicializado como NULL.

Parâmetros

linha	Linha da antena de destino.
coluna	Coluna da antena de destino.
frequencia	Frequência da antena de destino.

Retorna

Ponteiro para a adjacência criada, ou NULL em caso de falha de alocação.

Este é o diagrama das funções que utilizam esta função:

4.4.1.4 criarVertice()

```
Vertice * criarVertice (
            int linha,
```

```
            int coluna,
            char frequencia )
```

Cria dinamicamente uma nova antena (vértice) com os dados fornecidos.

O vértice criado representa uma antena posicionada numa determinada coordenada da matriz, com uma frequência associada. O campo de adjacências e o ponteiro para a próxima antena são inicializados como NULL.

Parâmetros

linha	Linha da matriz onde a antena está localizada.
coluna	Coluna da matriz onde a antena está localizada.
frequencia	Carácter que representa a frequência da antena.

Retorna

Ponteiro para o vértice criado, ou NULL em caso de falha de alocação.

Este é o diagrama das funções que utilizam esta função:

4.4.1.5 detetarIntersecoes()

```
bool detetarIntersecoes (
            Vertice * lista )
```

Verifica e imprime todas as interseções entre arestas do grafo.

Esta função percorre todas as arestas do grafo e verifica se há interseções entre elas, ignorando comparações duplicadas ou reversas.

Parâmetros

lista	Ponteiro para o início da lista de vértices (grafo).

Retorna

true se forem encontradas interseções, false caso contrário.

Grafo de chamadas desta função: Este é o diagrama das funções que utilizam esta função:

4.4.1.6 encontrarVertice()

```
Vertice * encontrarVertice (
            Vertice * lista,
            int linha,
            int coluna )
```

Procura um vértice(antena) na lista ligada com base nas suas coordenadas.

Percorre a lista ligada de vértices até encontrar uma antena cujas coordenadas (linha e coluna) coincidam com as fornecidas.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.
linha	Linha da matriz a procurar.
coluna	Coluna da matriz a procurar.

Retorna

Ponteiro para o vértice correspondente, ou NULL se não for encontrado.

Este é o diagrama das funções que utilizam esta função:

4.4.1.7 imprimirInterferencias()

```
bool imprimirInterferencias (
            Interferencia * lista )
```

Imprime todas as posições de interferência presentes na lista ligada.

Percorre a lista de interferências e imprime na consola as coordenadas (linha, coluna) de cada uma. Caso a lista esteja vazia, não imprime nada.

Parâmetros

lista	Ponteiro para o início da lista ligada de interferências.

Retorna

true se pelo menos uma interferência foi impressa, false se a lista estiver vazia.

Este é o diagrama das funções que utilizam esta função:

4.4.1.8 inserirAdjacente()

```
Adjacencia * inserirAdjacente (
            Adjacencia * listaAdj,
            Adjacencia * nova )
```

Insere uma adjacência no início da lista de adjacências do vértice.

Parâmetros

listaAdj	Ponteiro para a lista atual de adjacências.
nova	Ponteiro para a nova adjacência a inserir.

Retorna

Ponteiro para o novo início da lista de adjacências.

Este é o diagrama das funções que utilizam esta função:

4.4.1.9 inserirVertice()

```
Vertice * inserirVertice (
            Vertice * lista,
            Vertice * novo )
```

Insere um vértice no início da lista ligada de vértices.

Parâmetros

lista	Ponteiro para o início atual da lista ligada.
novo	Ponteiro para o novo vértice a ser inserido.

Retorna

Ponteiro para o novo início da lista.

Este é o diagrama das funções que utilizam esta função:

4.4.1.10 intersecoesParametricas()

```
bool intersecoesParametricas (
            int x1,
            int y1,
            int x2,
            int y2,
            int x3,
            int y3,
            int x4,
            int y4,
            double * ix,
            double * iy )
```

Verifica se dois segmentos se intersectam e, em caso afirmativo, calcula o ponto de interseção usando equações paramétricas.

Parâmetros

x1,y1	Coordenadas iniciais do primeiro segmento.
x2,y2	Coordenadas finais do primeiro segmento.
x3,y3	Coordenadas iniciais do segundo segmento.
x4,y4	Coordenadas finais do segundo segmento.
ix	Ponteiro para armazenar a coordenada X do ponto de interseção.
iy	Ponteiro para armazenar a coordenada Y do ponto de interseção.

Retorna

true se os dois segmentos se intersectam dentro de seus comprimentos, false caso contrário (paralelos, colineares ou extrapolados).

Este é o diagrama das funções que utilizam esta função:

4.4.1.11 libertarGrafo()

```
bool libertarGrafo (
            Vertice * lista )
```

Liberta toda a memória associada à lista ligada de vértices e suas adjacências.

Parâmetros

lista	Ponteiro para o início da lista de vértices.

Retorna

true se pelo menos um vértice foi libertado, false se a lista estava vazia.

Este é o diagrama das funções que utilizam esta função:

4.4.1.12 libertarInterferencias()

```
bool libertarInterferencias (
            Interferencia * lista )
```

Liberta toda a memória alocada para a lista ligada de interferências.

Esta função percorre a lista de interferências e liberta cada elemento, evitando vazamentos de memória. Após a execução, todos os nós da lista estarão desalocados.

Parâmetros

lista	Ponteiro para o início da lista ligada de interferências.

Retorna

true se pelo menos um elemento foi libertado, false se a lista era NULL.

Este é o diagrama das funções que utilizam esta função:

4.4.1.13 ligarAntenasMesmaFrequencia()

```
bool ligarAntenasMesmaFrequencia (
            Vertice * lista )
```

Liga vértices que têm a mesma frequência, criando adjacências entre eles.

Esta função percorre todos os pares de vértices e, quando encontra dois vértices com a mesma frequência, cria uma adjacência do primeiro para o segundo. As ligações são unidirecionais.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.

Retorna

true se pelo menos uma ligação foi criada, false se nenhuma ligação foi feita ou em caso de erro.

Grafo de chamadas desta função: Este é o diagrama das funções que utilizam esta função:

4.4.1.14 listarAntenas()

```
bool listarAntenas (
            Vertice * lista )
```

Lista todas as antenas presentes na lista ligada de vértices.

Esta função percorre a lista ligada de vértices (antenas) e imprime para a consola as suas coordenadas (linha, coluna) e frequência. Caso a lista esteja vazia, imprime uma mensagem informativa.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.

Retorna

true se pelo menos uma antena foi listada, false se a lista estiver vazia.

Este é o diagrama das funções que utilizam esta função:

4.5 Referência ao ficheiro grafo_funcoes.h

Declaração das funções de manipulação da lista ligada de vértices.

```
#include <stdbool.h>
#include "grafo_structs.h"
```

Diagrama de dependências de inclusão para grafo_funcoes.h: Este grafo mostra quais são os ficheiros que incluem directamente ou indirectamente este ficheiro:

Funções

- Vertice * criarVertice (int linha, int coluna, char frequencia)

  Cria dinamicamente uma nova antena (vértice) com os dados fornecidos.
- Vertice * inserirVertice (Vertice *lista, Vertice *novo)

  Insere um vértice no início da lista ligada de vértices.
- Vertice * encontrarVertice (Vertice *lista, int linha, int coluna)

  Procura um vértice(antena) na lista ligada com base nas suas coordenadas.

- int contarVertices (Vertice ∗lista)

  Conta o número total de vértices numa lista ligada.
- Adjacencia ∗ criarAdjacente (int linha, int coluna, char frequencia)

  Cria dinamicamente uma nova adjacência para um vértice.
- Adjacencia ∗ inserirAdjacente (Adjacencia ∗listaAdj, Adjacencia ∗nova)

  Insere uma adjacência no início da lista de adjacências do vértice.
- bool listarAntenas (Vertice ∗lista)

  Lista todas as antenas presentes na lista ligada de vértices.
- Interferencia ∗ calcularInterferencias (Vertice ∗lista, int numLinhas, int numColunas)

  Calcula interferências entre pares de antenas com a mesma frequência.
- bool imprimirInterferencias (Interferencia ∗lista)

  Imprime todas as posições de interferência presentes na lista ligada.
- bool libertarInterferencias (Interferencia ∗lista)

  Liberta toda a memória alocada para a lista ligada de interferências.
- bool libertarGrafo (Vertice ∗lista)

  Liberta toda a memória associada à lista ligada de vértices e suas adjacências.
- bool intersecoesParametricas (int x1, int y1, int x2, int y2, int x3, int y3, int x4, int y4, double ∗ix, double ∗iy)

  Verifica se dois segmentos se intersectam e, em caso afirmativo, calcula o ponto de interseção usando equações paramétricas.
- bool detetarIntersecoes (Vertice ∗lista)

  Verifica e imprime todas as interseções entre arestas do grafo.
- bool ligarAntenasMesmaFrequencia (Vertice ∗lista)

  Liga vértices que têm a mesma frequência, criando adjacências entre eles.

4.5.1 Descrição detalhada

Declaração das funções de manipulação da lista ligada de vértices.

4.5.2 Documentação das funções

4.5.2.1 calcularInterferencias()

```
Interferencia * calcularInterferencias (
            Vertice * lista,
            int numLinhas,
            int numColunas )
```

Calcula interferências entre pares de antenas com a mesma frequência.

Esta função percorre todos os pares de vértices (antenas) na lista e, quando encontra duas antenas com a mesma frequência, calcula os pontos simétricos entre elas, que representam possíveis zonas de interferência. Apenas pontos dentro dos limites da matriz (definidos por numLinhas e numColunas) são considerados válidos.

As interferências detetadas são armazenadas numa lista ligada.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.
numLinhas	Número total de linhas da matriz original.
numColunas	Número total de colunas da matriz original.

Retorna

Ponteiro para o início da lista ligada de interferências, ou NULL se nenhuma interferência for encontrada.

Este é o diagrama das funções que utilizam esta função:

4.5.2.2 contarVertices()

```
int contarVertices (
            Vertice * lista )
```

Conta o número total de vértices numa lista ligada.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.

Retorna

Número de vértices encontrados na lista.

4.5.2.3 criarAdjacente()

```
Adjacencia * criarAdjacente (
            int linha,
            int coluna,
            char frequencia )
```

Cria dinamicamente uma nova adjacência para um vértice.

Esta função representa a ligação entre duas antenas, armazenando as coordenadas e a frequência da antena de destino. O campo `prox` é inicializado como NULL.

Parâmetros

linha	Linha da antena de destino.
coluna	Coluna da antena de destino.
frequencia	Frequência da antena de destino.

Retorna

Ponteiro para a adjacência criada, ou NULL em caso de falha de alocação.

Este é o diagrama das funções que utilizam esta função:

4.5.2.4 criarVertice()

```
Vertice * criarVertice (
            int linha,
```

```
            int coluna,
            char frequencia )
```

Cria dinamicamente uma nova antena (vértice) com os dados fornecidos.

O vértice criado representa uma antena posicionada numa determinada coordenada da matriz, com uma frequência associada. O campo de adjacências e o ponteiro para a próxima antena são inicializados como NULL.

Parâmetros

linha	Linha da matriz onde a antena está localizada.
coluna	Coluna da matriz onde a antena está localizada.
frequencia	Carácter que representa a frequência da antena.

Retorna

Ponteiro para o vértice criado, ou NULL em caso de falha de alocação.

Este é o diagrama das funções que utilizam esta função:

4.5.2.5 detetarIntersecoes()

```
bool detetarIntersecoes (
            Vertice * lista )
```

Verifica e imprime todas as interseções entre arestas do grafo.

Esta função percorre todas as arestas do grafo e verifica se há interseções entre elas, ignorando comparações duplicadas ou reversas.

Parâmetros

lista	Ponteiro para o início da lista de vértices (grafo).

Retorna

true se forem encontradas interseções, false caso contrário.

Grafo de chamadas desta função: Este é o diagrama das funções que utilizam esta função:

4.5.2.6 encontrarVertice()

```
Vertice * encontrarVertice (
            Vertice * lista,
            int linha,
            int coluna )
```

Procura um vértice(antena) na lista ligada com base nas suas coordenadas.

Percorre a lista ligada de vértices até encontrar uma antena cujas coordenadas (linha e coluna) coincidam com as fornecidas.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.
linha	Linha da matriz a procurar.
coluna	Coluna da matriz a procurar.

Retorna

Ponteiro para o vértice correspondente, ou NULL se não for encontrado.

Este é o diagrama das funções que utilizam esta função:

4.5.2.7 imprimirInterferencias()

```
bool imprimirInterferencias (
            Interferencia * lista )
```

Imprime todas as posições de interferência presentes na lista ligada.

Percorre a lista de interferências e imprime na consola as coordenadas (linha, coluna) de cada uma. Caso a lista esteja vazia, não imprime nada.

Parâmetros

lista	Ponteiro para o início da lista ligada de interferências.

Retorna

true se pelo menos uma interferência foi impressa, false se a lista estiver vazia.

Este é o diagrama das funções que utilizam esta função:

4.5.2.8 inserirAdjacente()

```
Adjacencia * inserirAdjacente (
            Adjacencia * listaAdj,
            Adjacencia * nova )
```

Insere uma adjacência no início da lista de adjacências do vértice.

Parâmetros

listaAdj	Ponteiro para a lista atual de adjacências.
nova	Ponteiro para a nova adjacência a inserir.

Retorna

Ponteiro para o novo início da lista de adjacências.

Este é o diagrama das funções que utilizam esta função:

4.5.2.9 inserirVertice()

```
Vertice * inserirVertice (
            Vertice * lista,
            Vertice * novo )
```

Insere um vértice no início da lista ligada de vértices.

Parâmetros

lista	Ponteiro para o início atual da lista ligada.
novo	Ponteiro para o novo vértice a ser inserido.

Retorna

Ponteiro para o novo início da lista.

Este é o diagrama das funções que utilizam esta função:

4.5.2.10 intersecoesParametricas()

```
bool intersecoesParametricas (
            int x1,
            int y1,
            int x2,
            int y2,
            int x3,
            int y3,
            int x4,
            int y4,
            double * ix,
            double * iy )
```

Verifica se dois segmentos se intersectam e, em caso afirmativo, calcula o ponto de interseção usando equações paramétricas.

Parâmetros

x1,y1	Coordenadas iniciais do primeiro segmento.
x2,y2	Coordenadas finais do primeiro segmento.
x3,y3	Coordenadas iniciais do segundo segmento.
x4,y4	Coordenadas finais do segundo segmento.
ix	Ponteiro para armazenar a coordenada X do ponto de interseção.
iy	Ponteiro para armazenar a coordenada Y do ponto de interseção.

Retorna

true se os dois segmentos se intersectam dentro de seus comprimentos, false caso contrário (paralelos, colineares ou extrapolados).

Este é o diagrama das funções que utilizam esta função:

4.5.2.11 libertarGrafo()

```
bool libertarGrafo (
            Vertice * lista )
```

Liberta toda a memória associada à lista ligada de vértices e suas adjacências.

Parâmetros

lista	Ponteiro para o início da lista de vértices.

Retorna

true se pelo menos um vértice foi libertado, false se a lista estava vazia.

Este é o diagrama das funções que utilizam esta função:

4.5.2.12 libertarInterferencias()

```
bool libertarInterferencias (
            Interferencia * lista )
```

Liberta toda a memória alocada para a lista ligada de interferências.

Esta função percorre a lista de interferências e liberta cada elemento, evitando vazamentos de memória. Após a execução, todos os nós da lista estarão desalocados.

Parâmetros

lista	Ponteiro para o início da lista ligada de interferências.

Retorna

true se pelo menos um elemento foi libertado, false se a lista era NULL.

Este é o diagrama das funções que utilizam esta função:

4.5.2.13 ligarAntenasMesmaFrequencia()

```
bool ligarAntenasMesmaFrequencia (
            Vertice * lista )
```

Liga vértices que têm a mesma frequência, criando adjacências entre eles.

Esta função percorre todos os pares de vértices e, quando encontra dois vértices com a mesma frequência, cria uma adjacência do primeiro para o segundo. As ligações são unidirecionais.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.

Retorna

true se pelo menos uma ligação foi criada, false se nenhuma ligação foi feita ou em caso de erro.

Grafo de chamadas desta função: Este é o diagrama das funções que utilizam esta função:

4.5.2.14 listarAntenas()

```
bool listarAntenas (
            Vertice * lista )
```

Lista todas as antenas presentes na lista ligada de vértices.

Esta função percorre a lista ligada de vértices (antenas) e imprime para a consola as suas coordenadas (linha, coluna) e frequência. Caso a lista esteja vazia, imprime uma mensagem informativa.

Parâmetros

lista	Ponteiro para o início da lista ligada de vértices.

Retorna

true se pelo menos uma antena foi listada, false se a lista estiver vazia.

Este é o diagrama das funções que utilizam esta função:

4.6 grafo_funcoes.h

Ir para a documentação deste ficheiro.

```
00001
00006 #ifndef GRAFO_FUNCOES_H
00007 #define GRAFO_FUNCOES_H
00008
00009 #include <stdbool.h>
00010 #include "grafo_structs.h"
00011
00024 Vertice* criarVertice(int linha, int coluna, char frequencia);
00025
00033 Vertice* inserirVertice(Vertice* lista, Vertice* novo);
00034
00046 Vertice* encontrarVertice(Vertice* lista, int linha, int coluna);
00047
00054 int contarVertices(Vertice* lista);
00055
00068 Adjacencia* criarAdjacente(int linha, int coluna, char frequencia);
00069
00077 Adjacencia* inserirAdjacente(Adjacencia* listaAdj, Adjacencia* nova);
00078
00089 bool listarAntenas(Vertice* lista);
00090
00107 Interferencia* calcularInterferencias(Vertice* lista, int numLinhas, int numColunas);
00108
00118 bool imprimirInterferencias(Interferencia* lista);
```

```
00119
00130 bool libertarInterferencias(Interferencia* lista);
00131
00138 bool libertarGrafo(Vertice* lista);
00139
00153 bool intersecoesParametricas(int x1, int y1, int x2, int y2, int x3, int y3, int x4, int y4, double*
      ix, double* iy);
00154
00164 bool detetarIntersecoes(Vertice* lista);
00165
00176 bool ligarAntenasMesmaFrequencia(Vertice* lista);
00177
00178 #endif
```

4.7 Referência ao ficheiro grafo_structs.h

Definição das estruturas de dados usadas para representar grafos.

Este grafo mostra quais são os ficheiros que incluem directamente ou indirectamente este ficheiro:

Estruturas de Dados

- struct Adjacencia

  Representa uma ligação entre vértices (antenas).
- struct Vertice

  Representa um vértice (antena) da lista principal.
- struct Interferencia

  Representa uma posição de interferência detetada.

Definições de tipos

- typedef ∗struct Adjacencia Adjacencia
- typedef struct Vertice Vertice
- typedef struct Interferencia Interferencia

4.7.1 Descrição detalhada

Definição das estruturas de dados usadas para representar grafos.

Contém as estruturas base como `Vertice`, `Adjacencia` e `Interferencia`, utilizadas na representação do grafo e suas conexões.

Autor

Filipe

Data

Maio 2025

4.7.2 Documentação dos tipos

4.7.2.1 Adjacencia

```
typedef * struct Adjacencia Adjacencia
```

4.7.2.2 Interferencia

```
typedef struct Interferencia Interferencia
```

4.7.2.3 Vertice

```
typedef struct Vertice Vertice
```

4.8 grafo_structs.h

Ir para a documentação deste ficheiro.

```
00001
00011 #ifndef GRAFO_STRUCTS_H
00012 #define GRAFO_STRUCTS_H
00013
00019  */
00020 typedef struct Adjacencia {
00021     int linha, coluna;
00022     char frequencia;
00023     struct Adjacencia* prox;
00024 } Adjacencia;
00025
00030 typedef struct Vertice {
00031     int linha, coluna;
00032     char frequencia;
00033     Adjacencia* adjacentes;
00034     struct Vertice* prox;
00035 } Vertice;
00036
00041 typedef struct Interferencia {
00042     int linha;
00043     int coluna;
00044     struct Interferencia* prox;
00045 } Interferencia;
00046
00047 #endif
```

4.9 Referência ao ficheiro main.c

Ficheiro principal de execução do programa de análise de grafos de antenas.

```
#include <stdio.h>
#include <stdlib.h>
#include "grafo_funcoes.h"
#include "ficheiros.h"
```

Diagrama de dependências de inclusão para main.c:

Funções

- int main ()

  Função principal do programa.

4.9.1 Descrição detalhada

Ficheiro principal de execução do programa de análise de grafos de antenas.

Utiliza a biblioteca de grafos para carregar os dados, ligar antenas, detetar interferências e interseções, e aplicar algoritmos de procura.

Autor

Filipe

Data

Maio 2025

4.9.2 Documentação das funções

4.9.2.1 main()

```
int main ( )
```

Função principal do programa.

Esta função inicializa a estrutura de grafo, carrega os dados das antenas a partir de ficheiros, calcula interferências e interseções, e apresenta os resultados.

Retorna

int Código de retorno do programa (0 em caso de sucesso).

Grafo de chamadas desta função:

4.10 Referência ao ficheiro procura.c

Implementação de DFS e BFS para listas ligadas.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "grafo_funcoes.h"
```

Diagrama de dependências de inclusão para procura.c:

Funções

- bool dfs (Vertice *lista, int linha, int coluna, bool *visitado, int total)

  Executa pesquisa em profundidade (DFS) a partir de coordenadas.
- bool bfs (Vertice *lista, int linha, int coluna, int total)

  Executa pesquisa em largura (BFS) a partir de coordenadas.

4.10.1 Descrição detalhada

Implementação de DFS e BFS para listas ligadas.

4.10.2 Documentação das funções

4.10.2.1 bfs()

```
bool bfs (
            Vertice * lista,
            int linha,
            int coluna,
            int total )
```

Executa pesquisa em largura (BFS) a partir de coordenadas.

Parâmetros

lista	Lista ligada de vértices.
linha	Linha do vértice de origem.
coluna	Coluna do vértice de origem.
total	Total de vértices no grafo.

Retorna

true se a BFS for executada, false caso contrário.

Grafo de chamadas desta função:

4.10.2.2 dfs()

```
bool dfs (
            Vertice * lista,
            int linha,
            int coluna,
            bool * visitado,
            int total )
```

Executa pesquisa em profundidade (DFS) a partir de coordenadas.

Parâmetros

lista	Lista ligada de vértices.
linha	Linha do vértice de origem.
coluna	Coluna do vértice de origem.
visitado	Array de booleanos indicando vértices visitados.
total	Total de vértices no grafo.

Retorna

true se a DFS for executada, false caso contrário.

Grafo de chamadas desta função: Este é o diagrama das funções que utilizam esta função:

4.11 Referência ao ficheiro procura.h

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "grafo_funcoes.h"
```

Diagrama de dependências de inclusão para procura.h:

Funções

- bool dfs (Vertice *lista, int linha, int coluna, bool *visitado, int total)

  Executa pesquisa em profundidade (DFS) a partir de coordenadas.
- bool bfs (Vertice *lista, int linha, int coluna, int total)

  Executa pesquisa em largura (BFS) a partir de coordenadas.

4.11.1 Documentação das funções

4.11.1.1 bfs()

```
bool bfs (
            Vertice * lista,
            int linha,
            int coluna,
            int total )
```

Executa pesquisa em largura (BFS) a partir de coordenadas.

Parâmetros

lista	Lista ligada de vértices.
linha	Linha do vértice de origem.
coluna	Coluna do vértice de origem.
total	Total de vértices no grafo.

Retorna

true se a BFS for executada, false caso contrário.

Grafo de chamadas desta função:

4.11.1.2 dfs()

```
bool dfs (
            Vertice * lista,
            int linha,
            int coluna,
            bool * visitado,
            int total )
```

Executa pesquisa em profundidade (DFS) a partir de coordenadas.

Parâmetros

lista	Lista ligada de vértices.
linha	Linha do vértice de origem.
coluna	Coluna do vértice de origem.
visitado	Array de booleanos indicando vértices visitados.
total	Total de vértices no grafo.

Retorna

true se a DFS for executada, false caso contrário.

Grafo de chamadas desta função: Este é o diagrama das funções que utilizam esta função:

4.12 procura.h

Ir para a documentação deste ficheiro.

```
00001
00006 #include <stdio.h>
00007 #include <stdlib.h>
00008 #include <string.h>
00009 #include "grafo_funcoes.h"
00010
00021 bool dfs(Vertice* lista, int linha, int coluna, bool* visitado, int total);
00022
00032 bool bfs(Vertice* lista, int linha, int coluna, int total);
```

Índice

Adjacencia, 5
  coluna, 5
  frequencia, 5
  grafo_structs.h, 29
  linha, 5
  prox, 6
adjacentes
  Vertice, 7

bfs
  procura.c, 31
  procura.h, 32

calcularInterferencias
  grafo_funcoes.c, 12
  grafo_funcoes.h, 20
carregarMatriz
  ficheiros.h, 10
coluna
  Adjacencia, 5
  Interferencia, 6
  Vertice, 7
contarVertices
  grafo_funcoes.c, 13
  grafo_funcoes.h, 21
criarAdjacente
  grafo_funcoes.c, 13
  grafo_funcoes.h, 21
criarVertice
  grafo_funcoes.c, 13
  grafo_funcoes.h, 21

detetarIntersecoes
  grafo_funcoes.c, 14
  grafo_funcoes.h, 22
dfs
  procura.c, 31
  procura.h, 32

encontrarVertice
  grafo_funcoes.c, 14
  grafo_funcoes.h, 22

ficheiros.c, 9
ficheiros.h, 9
  carregarMatriz, 10
  gravarGrafoBincomInterferencias, 10
  lerGrafoBinario, 11
frequencia
  Adjacencia, 5
  Vertice, 7

grafo_funcoes.c, 11
  calcularInterferencias, 12
  contarVertices, 13
  criarAdjacente, 13
  criarVertice, 13
  detetarIntersecoes, 14
  encontrarVertice, 14
  imprimirInterferencias, 15
  inserirAdjacente, 15
  inserirVertice, 16
  intersecoesParametricas, 16
  libertarGrafo, 17
  libertarInterferencias, 17
  ligarAntenasMesmaFrequencia, 17
  listarAntenas, 19
grafo_funcoes.h, 19
  calcularInterferencias, 20
  contarVertices, 21
  criarAdjacente, 21
  criarVertice, 21
  detetarIntersecoes, 22
  encontrarVertice, 22
  imprimirInterferencias, 23
  inserirAdjacente, 23
  inserirVertice, 24
  intersecoesParametricas, 24
  libertarGrafo, 25
  libertarInterferencias, 25
  ligarAntenasMesmaFrequencia, 25
  listarAntenas, 27
grafo_structs.h, 28
  Adjacencia, 29
  Interferencia, 29
  Vertice, 29
gravarGrafoBincomInterferencias
  ficheiros.h, 10

imprimirInterferencias
  grafo_funcoes.c, 15
  grafo_funcoes.h, 23
inserirAdjacente
  grafo_funcoes.c, 15
  grafo_funcoes.h, 23
inserirVertice
  grafo_funcoes.c, 16
  grafo_funcoes.h, 24
Interferencia, 6
  coluna, 6

grafo_structs.h, 29
linha, 6
prox, 6
intersecoesParametricas
grafo_funcoes.c, 16
grafo_funcoes.h, 24

lerGrafoBinario
ficheiros.h, 11
libertarGrafo
grafo_funcoes.c, 17
grafo_funcoes.h, 25
libertarInterferencias
grafo_funcoes.c, 17
grafo_funcoes.h, 25
ligarAntenasMesmaFrequencia
grafo_funcoes.c, 17
grafo_funcoes.h, 25
linha
Adjacencia, 5
Interferencia, 6
Vertice, 7
listarAntenas
grafo_funcoes.c, 19
grafo_funcoes.h, 27

main
main.c, 30
main.c, 29
main, 30

procura.c, 30
bfs, 31
dfs, 31
procura.h, 32
bfs, 32
dfs, 32
prox
Adjacencia, 6
Interferencia, 6
Vertice, 7

Vertice, 7
adjacentes, 7
coluna, 7
frequencia, 7
grafo_structs.h, 29
linha, 7
prox, 7